

Arquitectura de Sistemas Colaborativos en Tiempo Real

Diseño e implementación de un motor de edición de texto plano distribuido, presencia concurrente y mensajería integrada.

Destinatario:	Estado: Propuesta de Diseño Avanzada	Latencia Objetivo: < 200 ms	Clasificación: Confidencial
Equipo de Desarrollo / Arquitectura		p95 (Cursores)	Técnico

1. Resumen Ejecutivo y Objetivos de Red

El presente informe describe el modelo arquitectónico óptimo para soportar una aplicación web de edición colaborativa en tiempo real de documentos de texto plano (v.g., código fuente). El sistema debe garantizar que múltiples usuarios distribuidos geográficamente visualicen de manera fluida las operaciones de texto de sus pares, la posición precisa de sus respectivos cursores, y puedan comunicarse concurrentemente a través de un chat contextual integrado.

Uno de los desafíos fundamentales es mitigar la degradación de la experiencia debido a la latencia en zonas remotas o periféricas, tales como regiones del extremo sur (v.g., Ushuaia, UTC-3), donde el tiempo de ida y vuelta físico (RTT) hacia los centros de datos continentales es intrínsecamente elevado. La meta de diseño impone un límite estricto para el procesamiento y propagación de estados efímeros:

$$\Delta t_{total} = RTT_{edge} + \Delta t_{proc} + \Delta t_{fanout} \leq 200\ ms\ (p95)$$

2. Persistencia y Ciclo de Vida del Documento: ¿Dónde "Vive"?

2.1. El Servidor como Fuente de Verdad Canónica

El documento no "vive" en los clientes; residirá de forma canónica en un servicio backend centralizado denominado **Document Service**. Los clientes mantienen copias o réplicas locales proyectadas directamente en la memoria del navegador (dentro del estado del editor).

- **Propiedad del Recurso:** El documento pertenece estrictamente a la entidad u organización que lo crea (v.g., cuenta corporativa, espacio de trabajo). El servicio valida cada conexión contra una lista de control de acceso (ACL) explícita y un registro de auditoría inmutable.
- **Estrategia de Evicción por Inactividad (Flushing):** Para evitar el desperdicio de memoria volátil en el cluster de backend, el documento se mantiene caliente en la memoria del *Collaboration Service* únicamente mientras existan usuarios interactuando con él. En el instante en que el contador de usuarios activos llega a cero, el sistema dispara un mecanismo de persistencia por evicción:
 1. Se consolida el estado final aplicando el log de operaciones pendiente.
 2. Se guarda un *snapshot* definitivo en un Object Storage robusto (v.g., compatible con S3).
 3. Se escribe el Write-Ahead Log (WAL) final en la base de datos relacional/documental.
 4. Se libera el documento de la memoria del servidor de colaboración.

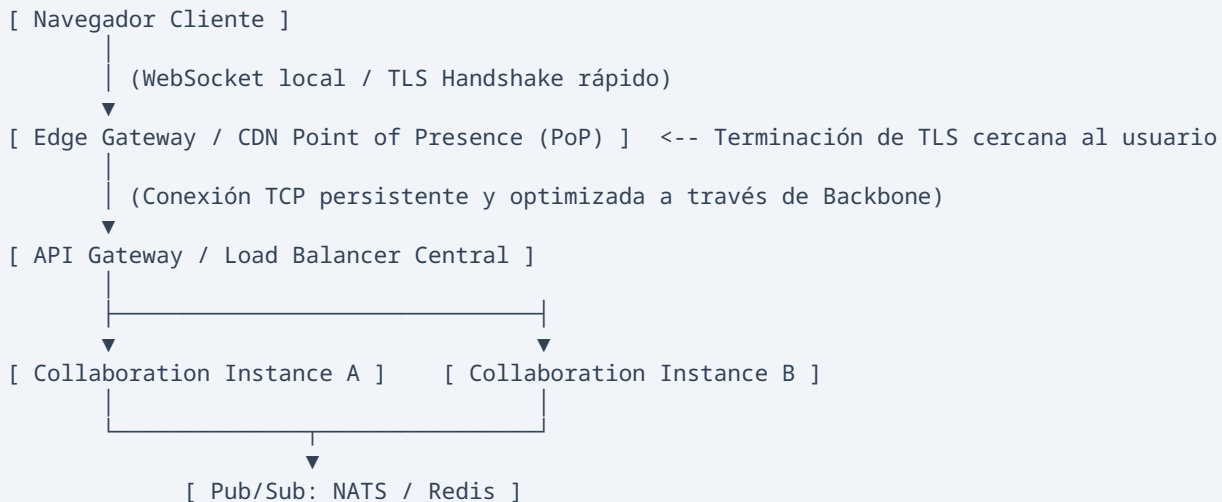
3. Modelo de Sincronización: El Paradigma de Concurrency

Históricamente se ha planteado una falsa dicotomía estricta: Transformación Operacional (OT) para servidores centralizados y Tipos de Datos Replicados Libres de Conflictos (CRDT) para entornos puramente Peer-to-Peer (P2P). En la ingeniería de software contemporánea, la estrategia recomendada para un editor de texto/código plano es implementar un **CRDT Centralizado Autorizativo** (v.g., utilizando motores maduros como *Yjs* o *Automerge* corriendo directamente sobre procesos del servidor).

Criterio de Evaluación	Transformación Operacional (OT)	CRDT Centralizado Autorizativo
Complejidad de Código	Extremadamente alta. Requiere matrices de transformación complejas en el backend para manejar la concurrencia asimétrica. Propenso a <i>edge-cases</i> .	Baja/Moderada. La convergencia está garantizada matemáticamente por la estructura de datos por diseño. El servidor actúa como secuenciador y validador.
Manejo de Metadatos	Bajo. Solo transmite deltas puros (inserciones/eliminaciones) indexados por posición y versión. El historial es lineal.	Mayor volumen. Requiere identificadores únicos por carácter o bloque (IDs de usuario + contadores de secuencia), mitigado por algoritmos de compresión modernos.
Soporte Offline Complejo	Requiere algoritmos de <i>rebase</i> muy pesados si un usuario se desconecta por periodos prolongados y acumula cientos de operaciones.	Nativo y robusto. Las réplicas locales divergen temporalmente y convergen de forma exacta e idéntica en cuanto los deltas son retransmitidos.
Recomendación de Uso	Desaconsejado para desarrollos desde cero debido a los costos de mantenimiento y testing matemático de las funciones de transformación.	Selección Óptima. Permite usar librerías de alto rendimiento en el frontend (CodeMirror/Tiptap) integradas de forma nativa con el servidor.

4. Arquitectura de Conexión, Red y Topología Geográfica

Para cumplir con la meta de latencia <200 ms p95 desde zonas geográficamente aisladas, es inviable conectar el WebSocket directamente a un servidor central localizado en otra región continental sin intermediarios. La topología debe estructurarse en capas:



4.1. Terminación TLS en el Edge (Edge Termination)

El cliente negocia el protocolo WebSocket contra el nodo de red más cercano de la infraestructura distribuida (v.g., Cloudflare Workers o Gateways regionales). Esto garantiza que el apretón de manos criptográfico (TLS Handshake) se complete en pocos milisegundos, reduciendo drásticamente el impacto de la distancia en el establecimiento de la sesión. Luego, el Edge redirige el tráfico hacia el datacenter central mediante tuberías de red de largo alcance optimizadas.

5. Segregación de Canales de Comunicación y Mensajería

Es un error crítico de diseño transmitir el texto, la presencia y el chat por el mismo canal lógico sin priorización. Un flujo masivo de mensajes de chat o movimientos continuos del ratón no deben bloquear el procesamiento de la edición del código fuente (evitando el fenómeno de *Head-of-Line Blocking*).

Principio de Diseño: Aislamiento de Tópicos por Características de Datos

El backend debe implementar tres canales independientes sobre el transporte en tiempo real, cada uno respaldado por una infraestructura de distribución de datos adaptada a sus requerimientos específicos de durabilidad y consistencia.

1. Canal A: Operaciones de Texto (Deltas del Documento)

- **Naturaleza:** Crítico, secuencial, binario, requiere entrega garantizada y ordenamiento estricto.
- **Infraestructura Backend:** Los mensajes viajan compactados (v.g., Protocol Buffers o vectores Yjs binarios). Se procesan en una cola de Pub/Sub en memoria de ultra baja latencia como **NATS Core** o un cluster dedicado de **Redis Pub/Sub**. Una vez validados, se encolan asincrónicamente a un log durable de persistencia.

2. Canal B: Presencia (Cursores, Selecciones e Indicadores de Escritura)

- **Naturaleza:** Efímero, no confiable, de alta frecuencia. Si un paquete que indica la posición del cursor de un usuario se pierde, carece de importancia debido a que será reemplazado por la coordenada generada un milisegundo después.

- **Infraestructura Backend:** Se propaga utilizando una estrategia de *Fan-Out* directo sin confirmación de recepción (no ACKs). Viaja por canales lógicos paralelos o, de ser viable en el cliente, mediante canales de datos no confiables de WebRTC (Data Channels). El backend jamás almacena esto en base de datos.

3. Canal C: Chat de Coordinación Integrado

- **Naturaleza:** Durable, asíncrono, basado en texto estándar (JSON), baja frecuencia en comparación con las pulsaciones de teclas.
- **Infraestructura Backend:** Transmitido por el gateway e insertado de manera directa en la base de datos persistente (v.g., PostgreSQL o MongoDB). No interfiere con el bucle de renderizado en memoria viva del documento de código.

6. Diferenciación de Estados: "Viewing" vs "Working in the Moment"

Para escalar el sistema a miles de documentos concurrentes, el servidor debe optimizar agresivamente el ancho de banda consumido por la presencia. Se definen tres estados de ciclo de vida en la sesión de un cliente:

- **VIEWING (Activo Pasivo)** El usuario ha abierto el documento y lo está leyendo. Se establece la conexión WebSocket, pero funciona exclusivamente como un flujo de lectura (*Read-Only Stream*). El cliente recibe las actualizaciones del texto y los cursores de los demás, pero el navegador **no** emite eventos de presencia ni coordenadas de cursor, reduciendo la carga de procesamiento global a cero en el sentido de subida (upstream). El intervalo de *heartbeat* es largo (v.g., 30 segundos) solo para validar vitalidad de la red.
- **EDITING (Working in the Moment)** Se dispara de forma automática en el instante en que el usuario hace foco en el editor o presiona la primera tecla. El cliente cambia su estado internamente: activa una interfaz de usuario optimista (*Optimistic UI*), inicializa los buffers de operaciones locales no confirmadas (esperando el ACK correlativo del servidor) y empieza a emitir ráfagas de coordenadas de cursor y selección. El intervalo de presencia se eleva a alta frecuencia.
- **IDLE (Inactivo en Sesión)** Si el cliente permanece en modo **EDITING** pero el sistema detecta un periodo de inactividad de teclado/mouse superior a un umbral estricto ($t > 15$ segundos), un temporizador con técnica de *debounce* degrada de forma automática al usuario al estado de observación pasivo. Esto remueve inmediatamente su cursor de las pantallas de los demás usuarios concurrentes, liberando ancho de banda en la red distribuida.

7. Matriz de Decisiones Técnicas y Control de Riesgos

A modo de conclusión, la siguiente tabla resume los componentes críticos recomendados para mitigar los riesgos operativos del sistema y garantizar la estabilidad a largo plazo:

Componente / Desafío	Solución Elegida (Recomendada)	Justificación de Ingeniería e Impacto
Capa de Transporte Efímero	Gateways distribuidos + Pub/Sub vía NATS Core / Redis.	Evita el cuello de botella y la latencia de polling de herramientas pesadas como Kafka. Garantiza la entrega de cursores en tiempos sub-100ms dentro del cluster.
Manejo de Revocación de Permisos	Desconexión forzada (Kill Signal) vía canal de control.	Si se revoca el acceso a un usuario a mitad de sesión, el Document Service rechaza sus siguientes deltas y envía una señal al Gateway para destruir el WebSocket inmediatamente, impidiendo la inyección de código malicioso.
Estructura del Frente	Modelos agnósticos de componentes de alto rendimiento (v.g., TypeScript puro, Lit o CodeMirror nativo).	Minimiza las capas de abstracción y los ciclos de re-renderizado excesivos que introducen frameworks pesados de manipulación del DOM, sosteniendo tasas estables de 60 FPS en el pintado de cursores concurrentes.